

Projet Python — Groupe 4

Structures de données, complexité et récursivité

Enseignante : Rahma GARGOURI — 2026-2027

Présentation finale — Projet en groupe

Vous préparez une **présentation** (PowerPoint, PDF ou autre format) qui répond aux questions ci-dessous, accompagnée d'un **notebook Jupyter** contenant les solutions des exercices, à exécuter pendant la présentation.

Partie A — Structures de données, mutabilité et opérateurs

Questions

1. Pourquoi `list(set(liste))` ne convient-il pas pour supprimer les doublons d'une liste en conservant l'ordre ?
2. Quelle est la complexité d'un test d'appartenance (`x in ...`) dans une liste, et dans un ensemble (`set`) ?
3. À quel moment la valeur par défaut d'un argument de fonction est-elle créée ?
4. Quelle est la différence entre un opérateur logique (`and`, `or`, `not`) et un opérateur bit à bit (`&`, `|`, `^`, `~`) ? Dans quels cas donnent-ils exactement le même résultat, et dans quels cas un résultat différent ?

Exercices

A.1 — Compter les mots. Écrire `compter_mots(phrase)` qui renvoie un dictionnaire mot $\rightarrow$ nombre d'occurrences, puis refaire le calcul avec `collections.Counter`.

```
compter_mots("le taux monte le taux baisse le taux monte")
# {'le': 3, 'taux': 3, 'monte': 2, 'baisse': 1}
```

A.2 — Supprimer les doublons. Écrire `sans_doublons(liste)` qui supprime les doublons en conservant l'ordre.

```
sans_doublons([3, 1, 3, 2, 1]) # [3, 1, 2]
```

A.3 — Prédire l'affichage. Qu'affiche ce code ? Pourquoi ? Le corriger.

```
def ajouter(element, liste=[]):
    liste.append(element)
    return liste

print(ajouter(1))
print(ajouter(2))
```

A.4 — L'entier unique. Dans une liste d'entiers, chaque entier apparaît exactement deux fois, sauf un seul qui n'apparaît qu'une fois. Trouver cet entier en utilisant un **opérateur bit à bit**.

```
entier_unique([4, 1, 2, 1, 2]) # 4
```

Partie B — Complexité

Questions

1. Pourquoi la recherche dichotomique exige-t-elle une liste triée ?
2. Combien d'étapes au maximum faut-il pour une liste d'un million d'éléments ?
3. Quelle est la complexité en mémoire de votre version en $O(n)$ de l'exercice B.2 ?

Exercices

B.1 — Recherche dichotomique. Écrire `recherche_dichotomique(liste_triee, cible)` qui renvoie l'indice de `cible`, ou `-1`.

```
recherche_dichotomique([1, 3, 5, 7, 9, 11], 7)    # 3
```

B.2 — Meilleur gain. Une liste de taux est donnée dans l'ordre chronologique. Trouver le plus grand gain possible en achetant un jour puis en revendant un jour **ultérieur**. Écrire une version en $O(n^2)$ puis une en $O(n)$, et l'appliquer à la série réelle de vos taux. Si aucun gain positif n'est possible, renvoyer 0.

```
meilleur_gain([7, 1, 5, 3, 6, 4])    # 5 (acheter à 1, vendre à 6)
```

Partie C — Récursivité

Questions

1. Qu'est-ce que la pile d'appels ? Que s'y passe-t-il à chaque appel récursif ?
2. Que signifie l'erreur `RecursionError` ? Quelle est la profondeur maximale par défaut en Python, et comment la connaître ?

Exercices

C.1 — Tours de Hanoï. Le jeu (voir https://fr.wikipedia.org/wiki/Tours_de_Hano%C3%AF) se joue avec trois piquets A, B, C et n disques de tailles différentes, empilés au départ sur le piquet A, du plus grand (en bas) au plus petit (en haut). Le but est de déplacer tous les disques sur le piquet C, en respectant trois règles :

- on déplace un seul disque à la fois ;
- on ne peut prendre que le disque du dessus d'un piquet ;
- on ne pose jamais un disque sur un disque plus petit.

Idée récursive. Pour déplacer n disques de A vers C : on déplace d'abord les $n - 1$ disques du dessus de A vers B, puis le plus grand disque de A vers C, puis les $n - 1$ disques de B vers C.

Écrire une fonction récursive `hanoi(n, depart, arrivee, intermediaire)` qui affiche les déplacements à effectuer.

```
hanoi(2, "A", "C", "B")
# A -> B
# A -> C
# B -> C
```

Compter le nombre de déplacements pour $n = 1$ à 10. Quelle formule retrouvez-vous ? Quelle est la complexité ?